

Implementation Of A SIMPLE PWM GENERATOR For MICROCONTROLLER Using Verilog

AYUSH MALLIK

This project demonstrates how a simple and fast pulse width modulator (PWM) generator can be implemented using Verilog programming. It is simulated using ModelSim, a multi-language (hardware description language) simulation environment from Mentor Graphics and tested on Basys 2 FPGA development board from Digilent.

PWM generation

Block diagram of the PWM generator is shown in Fig. 1. Working principle of the generator is simple. It uses one counter and one comparator. The microcontroller unit provides 8-bit input into PWM module. Counter used in the PWM module is 8-bit. It increments its value on the positive edge of the clock (positive edge triggered). Comparator used in the PWM generator is also 8-bit.

Input given to PWM module is compared to the current value of the counter using the comparator. If current value of the counter is greater than the value given to the module as input, PWM output is pulled low. However, if current value of the counter is less than the value given as input to the module, output of PWM generator is pulled high.

Let us take a test case to understand the operation. Suppose, input to PWM module is 128 (10000000₂). Counter is initialised using Reset button, so that output is 0(00000000₂). During the first clock cycle, value of counter and input value to PWM module are compared.

The module finds that, value of counter is less than the value provided. This prompts the module to pull output of PWM generator high. This same paradigm keeps output of the

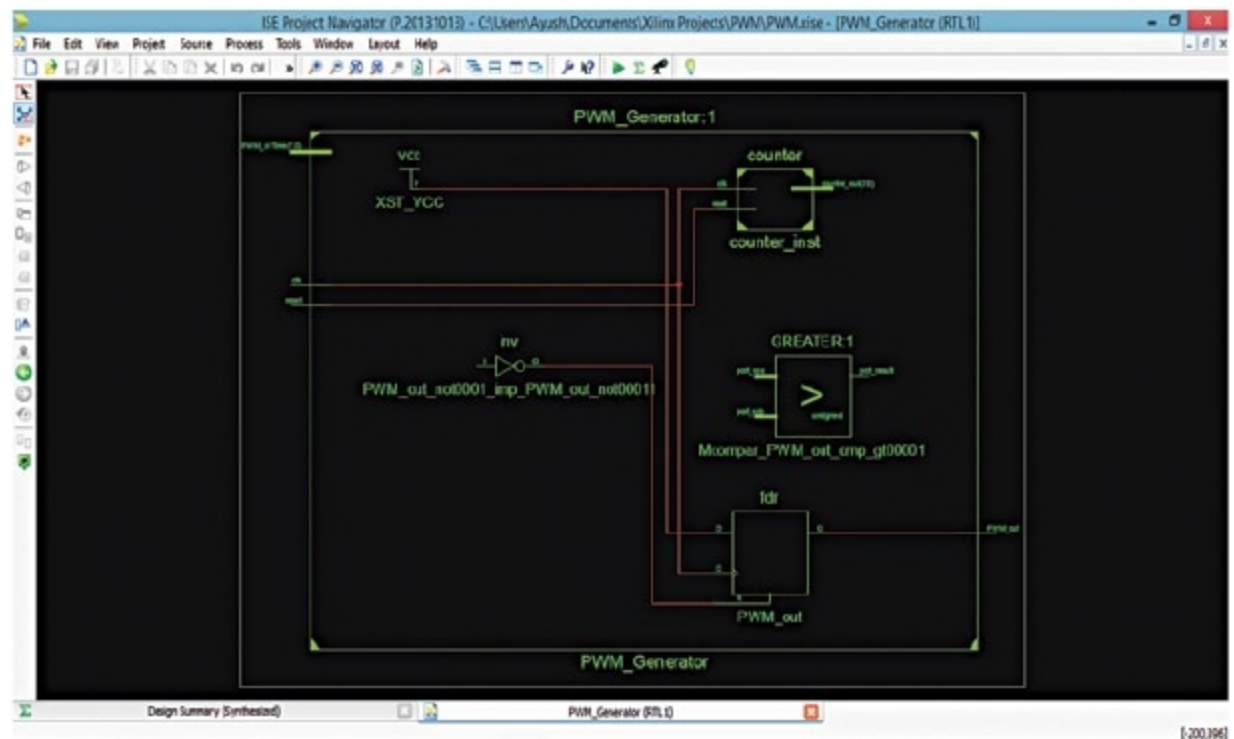


Fig. 1: Block diagram of PWM generator

generator pulled high until value of the counter reaches 128 (10000000₂).

In the next clock cycle, after the counter has reached 128, counter increments its value to 129 (10000001₂). Comparator finds that, current counter value is greater than the value provided to PWM module, therefore output must be pulled low.

Changing input to PWM module will consequently change the threshold value, where transition from high to low state occurs. Thus, PWM can be achieved by varying the input provided to PWM module.

Software program

Verilog main code used (PWM_main.v) for PWM generation is given below.

```
module PWM_generator (PWM_ontime, PWM_out ,
clk , reset);

input clk, reset; // Port type declared
input [7:0] PWM_ontime; // 8-bit PWM input
```

```
output reg PWM_out; // 1 bit PWM output
wire [7:0] counter_out; // 8-bit counter output

always @ (posedge clk)
begin
    if (PWM_ontime > counter_out)
        PWM_out <= 1;
    else
        PWM_out <= 0;
end

counter counter_inst(
    .clk (clk),
    .counter_out (counter_out),
    .reset(reset)
);

endmodule

module counter(counter_out,clk,reset);

output [7:0] counter_out;
input clk, reset;
reg [7:0] counter_out;
always @ (posedge clk)
    if (reset)
```